BAHIR DAR INSTITUTE OF TECHNOLOGY

Software Architecture And Design Lab Report

Name **ID**

Daniel Andualem……..1505662

Amhaselassie Mulugeta ……1505098

Title: Prototype Design Pattern

Introduction to Design Patterns

Design patterns are typical solutions to commonly occurring problems in software design. They represent best practices used by experienced object-oriented software developers. Among the creational design patterns, which deal with object creation mechanisms, the Prototype Design Pattern stands out as a highly efficient means of creating duplicates of existing objects while preserving their state.

Prototype Design Pattern Overview

The Prototype Design Pattern is a creational pattern that enables object creation by copying an existing object, known as the prototype. This pattern is particularly useful when the cost of creating a new object is more expensive than copying an existing one, or when creating an object involves a complex setup.

This design pattern involves

- A prototype interface that declares a clone() method.
- Concrete classes that implement this interface and define how the object should be copied.

The main goal is to reduce the complexity of creating objects and improve performance by avoiding the overhead of repeatedly instantiating and initializing objects from scratch.

Core Components of the Prototype Pattern

- *Prototype Interface* This interface declares the clone() method, which is responsible for returning a copy of the object that implements it.
- *Concrete Prototype Class* This class implements the Prototype interface and defines the logic for cloning itself.
- *Client* The client class interacts with the prototype interface to produce copies of objects without depending on their concrete classes.

Implementing the Prototype Design Pattern with EmployeeRecord Example

In the following implementation, the Prototype interface declares the clone() method. The EmployeeRecord class implements this interface and includes several attributes: name, salary, address, and id. It also includes a showRecord() method to display employee details.

```
// Prototype interface

interface Prototype {
```

```
    Prototype clone();
}
// Concrete class implementing the Prototype interface
class EmployeeRecord implements Prototype {
    private int id;
    private String name;
    private String address;
    private double salary;
    public EmployeeRecord(int id, String name, String address, double salary) {
        this.id = id;
        this.name = name;
        this.address = address;
        this.salary = salary;
    }
    // Cloning method
    @Override
    public Prototype clone() {
        return new EmployeeRecord(id, name, address, salary);
    }
    public void showRecord() {
        System.out.println("Employee ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Address: " + address);
        System.out.println("Salary: $" + salary);
```

```
        System.out.println("----------------------------");
    }
}
// Client Code
public class PrototypePatternDemo {
    public static void main(String[] args) {
        EmployeeRecord emp1 = new EmployeeRecord(101, "John Doe", "123 Main St", 75000);
        emp1.showRecord();
        EmployeeRecord emp2 = (EmployeeRecord) emp1.clone();
        emp2.showRecord();
    }
}
```

Benefits of Using Prototype Pattern

- *Performance* Creating a clone of an object is generally faster than creating a new instance.
- *Simplified Object Creation* Particularly useful when object initialization is expensive or complex.
- *Decouples Classes The* client does not need to know the exact class of the object it is cloning.
- *Runtime Object Configuration* Objects can be dynamically created and configured at runtime.

Real-World Applications

Graphic Editors When duplicating shapes or images.

Document Editing Software For copying document elements.

Game Development For copying game entities or characters with pre-configured settings.

Data Processing Systems Where large datasets or configurations are repeatedly used.

Conclusion

The Prototype Design Pattern is a powerful and flexible way to manage object creation. By using a prototype interface and implementing a clone method, developers can produce high-performing and easily maintainable applications. The EmployeeRecord example clearly demonstrates how to use this pattern effectively by encapsulating the clone logic and making the object duplication process straightforward.

By applying the Prototype Design Pattern, we can streamline object creation processes, avoid redundancy, and improve the overall performance and maintainability of applications.

Proxy Design Pattern

The Proxy Design Pattern provides a surrogate or placeholder object to control access to another object. This is useful when an object is resource-intensive or requires controlled access.

Example School Internet Access

In a school, the computer department restricts access to certain websites (e.g., Facebook, YouTube) during computer lectures. The ProxyInternet class acts as an intermediary, checking whether the requested website is on the restricted list before granting or denying access.

Components of Proxy Design Pattern

- *SchoolInternet Interface* Declares the `provideInternet()` method.
- *RealInternet Class* Implements `SchoolInternet` and provides unrestricted access while maintaining a department name attribute.
- *ProxyInternet Class* Implements `SchoolInternet`, checks for restricted websites, maintains department name, and delegates requests to `RealInternet` if allowed.

Implementation of Proxy Pattern

```
// Step 1: Create SchoolInternet interface
public interface SchoolInternet {
    void connectTo(String serverHost) throws Exception;
}
// Step 2: Create RealInternet class
public class RealInternet implements SchoolInternet {

   private String departmentName;

    public RealInternet(String departmentName) {
```

```
        this.departmentName = departmentName;
    }

    @Override
    public void connectTo(String Website) throws Exception {
        System.out.println("[" + departmentName + "] Connecting
to " + Website);


}
}
// Step 3: Create ProxyInternet class
public class ProxyInternet implements SchoolInternet {
     private String departmentName;
    private RealInternet realObject;

    private static List<String> bannedSites;
    private static List<String> allowedDepartments;

    static {
        bannedSites = new ArrayList<>();
        bannedSites.add("facebook.com");
        bannedSites.add("youtube.com");

        allowedDepartments = new ArrayList<>();
        allowedDepartments.add("Computer Science");
        allowedDepartments.add("IT");
        allowedDepartments.add("History");
    }

    public ProxyInternet(String departmentName) {
        this.departmentName = departmentName;
    }

    private boolean isDepartmentAllowed(String departmentName) {
        return allowedDepartments.contains(departmentName);
    }

    @Override
    public void connectTo(String Website) throws Exception {
        if (!isDepartmentAllowed(departmentName)) {
            throw new Exception("Access denied for department: "
+ departmentName);
        }

        if (bannedSites.contains(Website.toLowerCase())) {
```

```
            throw new Exception("Access denied to site: " +
Website);
        }

        realObject = new RealInternet(departmentName);
        realObject.connectTo(Website);
    }
}
// Step 4: Client class

public class ProxyDesignPatternClient {
public static void main(String[] args) {

try {
SchoolInternet internet1 = new    ProxyInternet("Computer
Science");
internet1.connectTo("geeksforgeeks.org"); // allowed
}catch (Exception e) {
System.out.println(e.getMessage()); }

try{

SchoolInternet internet2 = new ProxyInternet("History");
internet2.connectTo("wikipedia.org"); // denied by department

}catch (Exception e) {

 System.out.println(e.getMessage());
 }

 try{
SchoolInternet internet3 = new ProxyInternet("IT");
 internet3.connectTo("facebook.com"); // denied by site

}catch (Exception e) {

System.out.println(e.getMessage());
 }
```

```
    try{
   SchoolInternet internet4 = new ProxyInternet("software");
   internet4.connectTo("facebook.com"); // denied by site
    }catch (Exception e) {
    System.out.println(e.getMessage());
  }
  try{
  SchoolInternet internet5 = new ProxyInternet("IT");
  internet5.connectTo("example.com"); // denied by site
  }catch (Exception e) {
  System.out.println(e.getMessage());
      }
  try{
   SchoolInternet internet6 = new ProxyInternet("IT");
   internet6.connectTo("ddd.com"); // denied by site
   }
   catch (Exception e) {
   System.out.println(e.getMessage());
    }
     }
}
```

Benefits of Using Proxy Pattern

- *Access Control* Restricts access based on predefined rules.
- *Improved Performance* Defers object creation until necessary.
- *Security and Logging* Can be used for monitoring and authentication.
- *Lazy Initialization* Creates expensive objects only when required.

Conclusion

The Proxy Design Pattern is valuable when controlling access to an object is necessary. In the school internet example, the ProxyInternet class ensures that restricted websites are blocked while allowing legitimate internet access. This pattern enhances security, improves performance, and provides a structured approach to managing resource-intensive objects.

By understanding both Prototype and Proxy Design Patterns, developers can create more efficient and scalable software applications.